BİLGİSAYAR MİMARİSİ 2.HAFTA Notları

Arda Kumartaşlıoğlu

25-09-2025

1 Bilgisayar Mimari Tasarımı

1.1 Performans (Hız)

- **Amaç:** Bilgisayarı mümkün olduğunca hızlı yapmak.
- Nasıl yapılır?
  - Saat hızı artırılır (CPU daha hızlı çalışır).
  - Komutlar daha kısa sürede işlenir.
  - Pipeline (boru hattı) ve paralellik (çok çekirdek) kullanılır.
  - RAM erişimi hızlandırılır.

  Örneğin:
  Bir fabrika düşün. İşçiler hızlı, bant hızlı, malzeme de zamanında geliyorsa → üretim süper hızlı olur.

1.2 Verimlilik (Enerji ve Kaynak Kullanımı)

- **Amaç:** Az enerjiyle çok iş yapmak.
- Nasıl yapılır?
  - Az güç tüketen bileşenler seçilir.
  - Gereksiz işlemler azaltılır.
  - Isınmayı önlemek için işlemci gücü dengelenir (TDP).

  Örneğin:
  Bir araba düşün: Az benzinle çok yol gidiyorsa → verimli motordur. Bilgisayarda da az enerjiyle çok işlem yapan CPU verimlidir.

1.3 Ölçeklenebilirlik

- **Amaç:** Sistemin ihtiyaç arttıkça büyüyebilmesi.
- Nasıl yapılır?
  - Çok çekirdekli işlemci desteği olmalı.
  - Parçalar kolay eklenip çıkarılmalı (modüler tasarım).
  - Bellek ve veri yolu geniş olmalı.

  Örneğin:
  Bilgisayarına sonradan RAM, ekran kartı, işlemci ekleyebiliyorsan → ölçeklenebilir bir sistemdir.

1.4 Uyumluluk ve Standartlar

- **Amaç:** Farklı yazılımlar ve donanımların uyum içinde çalışması.
- Nasıl yapılır?
  - Yaygın komut setleri (örneğin x86, ARM) tercih edilir.
  - Eski yazılımlarla uyum sağlanır.

  Örneğin:
  Bir oyun Windows'ta, Linux'ta ve Mac'te çalışabiliyorsa → uyumluluk iyi tasarlanmıştır.

1.5 Güvenilirlik ve Hata Toleransı

- **Amaç:** Bilgisayar hata yapsa bile çökmemeli, kendini toparlamalı,
- Nasıl yapılır?
  - Hata algılama sistemleri (ECC RAM gibi)
  - Yedekleme sistemleri
  - Dayanıklı işlemci ve bellek

  Örneğin:
  Bir belgeyi kaydetmeyi unutsan bile otomatik kurtarma varsa → hata toleransı sağlanmış demektir.

1.6 Maliyet ve Üretilebilirlik

- **Amaç:** Hem ucuz üretmek hem de yaygınlaştırmak.
- Nasıl yapılır?
  - Fiyat-Performans dengesi gözetilir.
  - Ucuz ama güçlü bileşenler tercih edilir.

  Örneğin:
  Bir işlemci 5000TL ye satılıyorsa ama 3000TL lik olanla aynı işi yapıyorsa → iyi tasarlanmamıştır.

1.7 Kullanım Amacı ve Hedef Kitle

- **Amaç:** Bilgisayarı ne için kullanılacaksa ona göre tasarlamak.
- Nasıl yapılır?
  - Oyun bilgisayarı→ yüksek performans.
  - Sunucu → Güvenilir ve çok çekirdek.
  - Gömülü sistem (örneğin buzdolabı içindeki mikrodenetleyici)→ düşük güç tüketimi.

  Örneğin:
  Bir süper bilgisayarla buzdolabındaki işlemci aynı olmaz.

2 Donanım Seviyeleri

Seviye	Ne Anlama Gelir	Basit Açıklama
1. Donanım (Hardware)	Fiziksel parçalar (CPU, RAM, Disk, Ekran, Klavye)	Bilgisayarın somut, görünen kısmı
2. Mikroprogramlama	CPU içindeki **en küçük kontrol sinyalleri**	Komutları daha küçük adımlara böler
3. Makine Dili (Assembly)	CPU'nun **doğrudan anlayabildiği komutlar**	"ADD", "MOV", "JMP" gibi temel işlemler
4. Yüksek Seviyeli Diller	İnsanların kolay yazdığı diller (C, Python, Java)	Derleyiciyle makine diline çevrilir

Bilgisayar Dilleri ve Donanım Seviyeleri

3 Bilgisayar Mimarisinin 3 Temel Kavramı

Kavram	Ne yapar	Basit Açıklama
1. ISA (Instruction Set Architecture)	FiYazılım ile donanım arasındaki köprü	CPU hangi komutları anlayabilir, onu tanımlar
2. Microarchitecture	Komutların içeride nasıl işlendiğini belirler	Pipeline, cache, paralellik gibi şeyler burada belirlenir
3. System Design	Tüm sistemi kapsar (bellek, G/Ç, veri yolları)	Bilgisayarın genel "altyapı tasarımı"dır

3.1 Komut Kümesi Mimarisi (ISA) Türleri

- **ISA (Instruction Set Architecture)**
  - Yazılım ile donanım arasındaki köprü
  - CPU hangi komutları anlayabilir, onu tanımlar
- **Microarchitecture**
  - Komutların içeride nasıl işlendiğini belirler
  - Pipeline, cache, paralellik gibi şeyler burada belirlenir
- **System Design**
  - Tüm sistemi kapsar (bellek, G/Ç, veri yolları)
  - Bilgisayarın genel "altyapı tasarımı"dır

3.2 Komut Kümesi Mimarisi (ISA) Türleri

- **Stack (Yığın) Tabanlı**
  - Komutlar yığına konur, LIFO (son giren ilk çıkar) mantığıyla çalışır
  - Örneğin: Eski tip hesap makineleri
- **Accumulator (Toplayıcı) Tabanlı**
  - Tüm işlemler tek bir kayıt (register) üzerinden yapılır
  - Örneğin: (register) üzerinden yapılır İlk nesil işlemciler
- **Register (Kayıt) Tabanlı**
  - Birden fazla kayıtla işlem yapılır, en hızlısı
  - Örneğin: Modern CPU'lar (x86, ARM)

Aşama	Açıklama	Örnek
1. Fetch (Getir)	Komut bellekte bulunur ve alınır	"Toplama yap" komutu getirildi
2. Decode (Çöz)	Komutun ne olduğu çözülür	"Bu komut ADD (toplama)" denir
3. Execute (Yürüt)	ALU (aritmetik birim) işlemi yapar	5 + 3 hesaplanır
4. Write Back (Sonucu Yaz)	Sonuç belleğe veya kayda yazılır	8 sonucu RAM'e kaydedilir
5. Update PC (Sonraki Komuta Geç)	Program sayacı bir sonraki komuta geçer	Sıradaki komut işlenmeye hazırlanır

Komut Döngüsünün Aşamaları (Fetch–Decode–Execute)

Örnek:

```
x = 5 + 3
```

- Bellekten "+" komutu getirilir
- "+" işlemi olduğu anlaşılır
- 5 ve 3 toplanır
- Sonuç 8 olarak kaydedilir
- CPU sıradaki komuta geçer